Intelligenza Artificiale

“Solving problems by searching”

(part 2)

Nicola Bellotto

A.A. 2025/2026

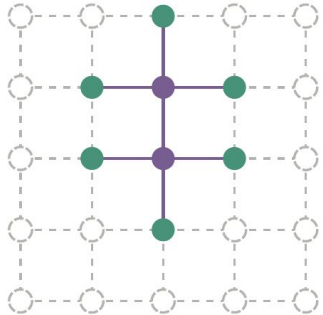
Lecture outline

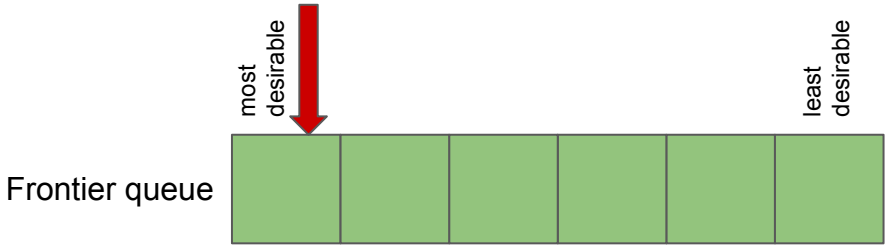
- Informed search
- Heuristic functions
- Local search

Slides partly based on Russell & Norvig instructors material http://aima.cs.berkeley.edu/instructors.html

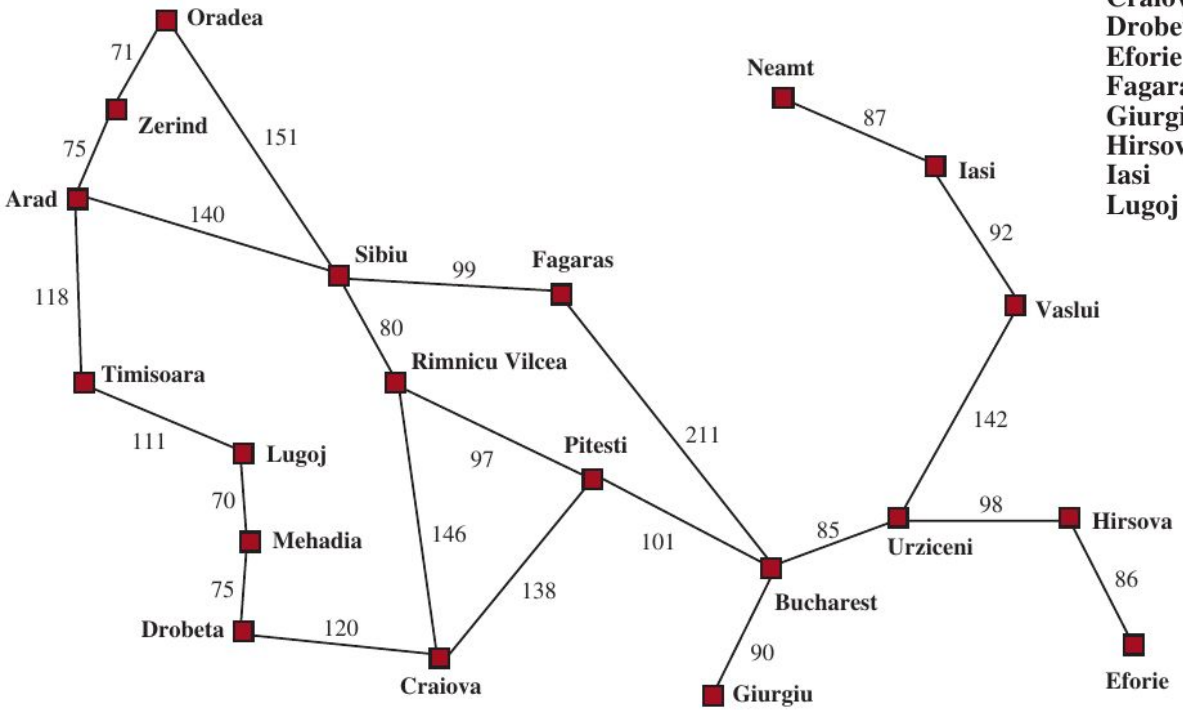
Informed search

- Idea: use an **heuristic function** for each node
  - estimate cost of path from node to goal
  - expand most desirable unexpanded node
- Implementation
  - The frontier is a queue sorted in decreasing order of desirability
  - Special cases
    - **Greedy search**
    - **A* search**





Romania with step costs in km

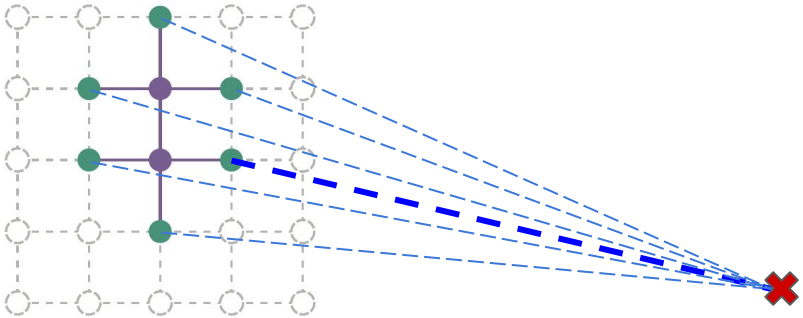


Straight-line distance to Bucharest

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

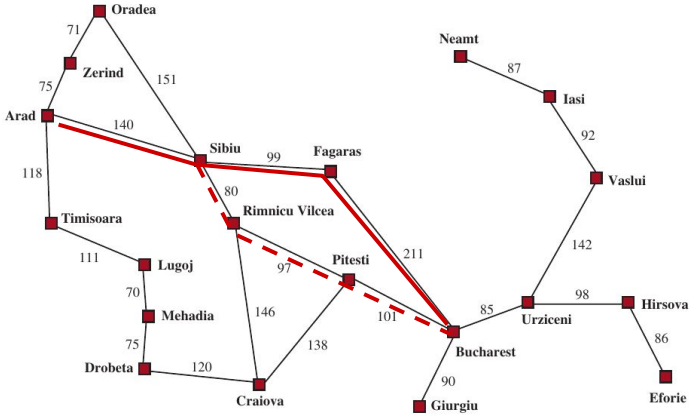
Greedy best-first search

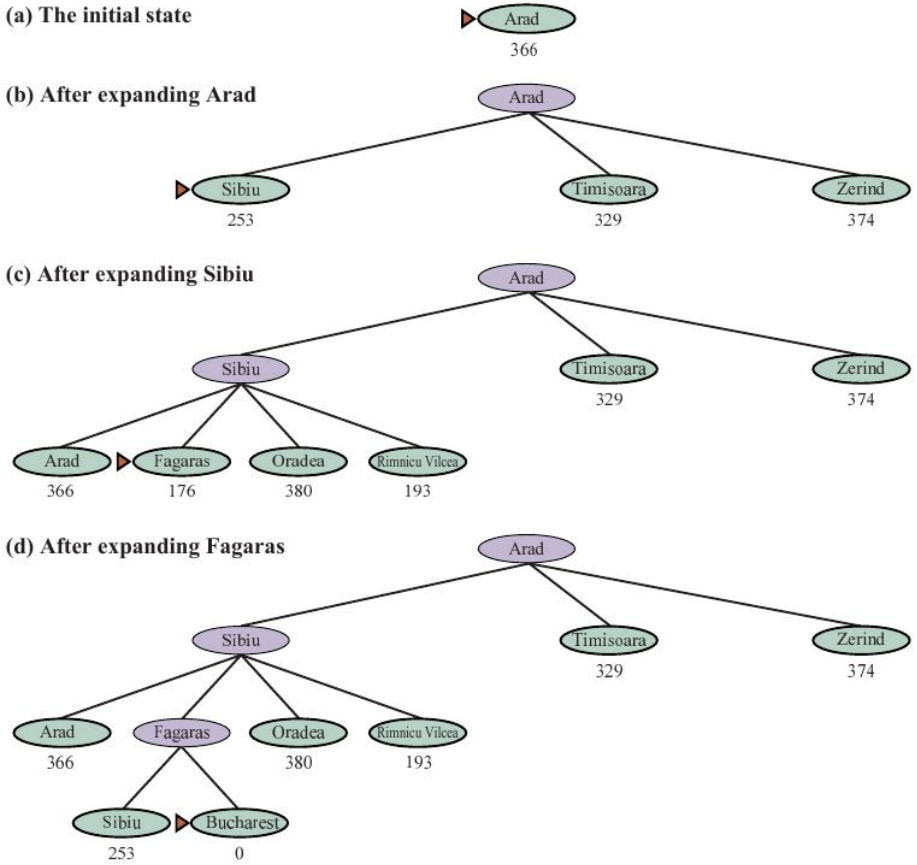
- Heuristic function $h(n)$ estimates of cost from n to the closest goal
- Greedy best-first search expands the node that **appears** to be closest to goal
  - E.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
  - Expand node with lowest distance



Greedy search example

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374





Properties of greedy best-first search

- Complete?
  - Only in finite state space, with no loops
- Time?
  - $O(b^m)$, but a good heuristic can improve a lot!
- Space?
  - $O(b^m)$ – keeps all nodes in memory
- Optimal?
  - **No**

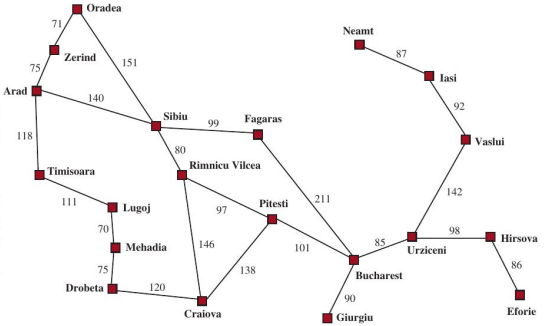
A* search

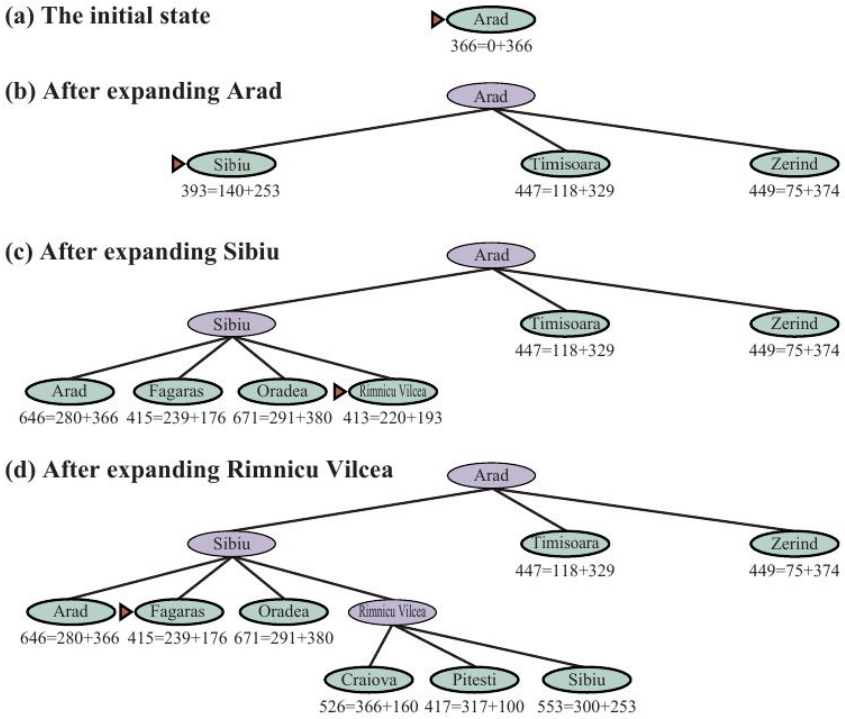
- IDEA: avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$
  - $g(n)$ = cost so far to reach n
  - $h(n)$ = heuristic, i.e. estimated cost to goal from n
  - $f(n)$ = estimated total cost through n

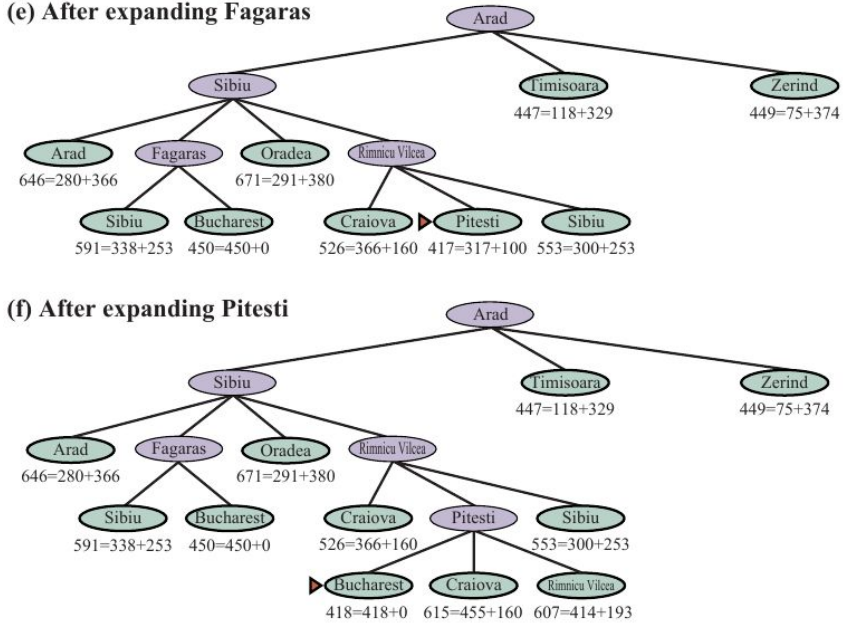
A* search example

Straight-line distance to Bucharest

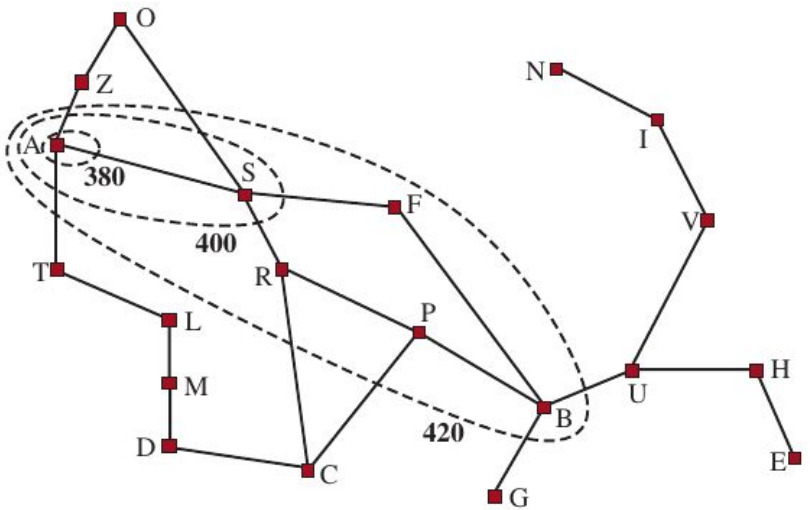
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374







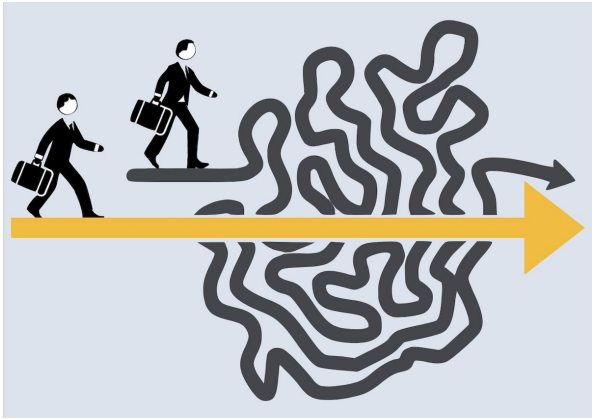
Search contours



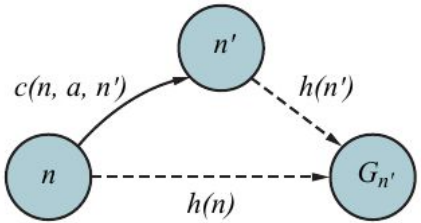
- A* expands nodes in order of increasing *f* values
- Gradually adds "*f*-contours" of nodes
- Contour *i* has all nodes with $f = f_i$, where $f_i < f_{i+1}$
- Differently from BFS contours, with A* their shape "tends" to the goal

Admissible heuristic

- A* search uses an **admissible heuristic**
  - $h(n) \geq 0$, and $h(G) = 0$ for any goal G
  - $h(n) \leq h^*(n)$ where $h^*(n)$ is the true cost from n
    - E.g., $h_{SLD}(n)$ never overestimates the actual road distance, so it is an admissible heuristic



  - If $h(n) \leq c(n, a, n') + h(n')$, then the heuristic is also **consistent** (or **monotone**)
    - consistent heuristic ⇒ admissible



Optimality of A*

- Suppose some suboptimal goal G_s has been generated, while n is an unexpanded node on a shortest path to the optimal goal G_o

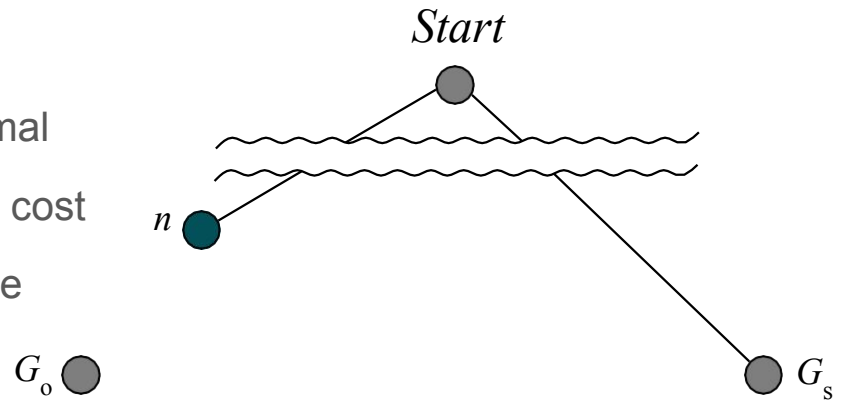
$f(G_s) = g(G_s)$ since $h(G_s) = 0$

$> g(G_o)$ since G_s is suboptimal

$= g(n) + h^*(n)$ where h^* is the true cost

$\geq g(n) + h(n)$ since h is admissible

$= f(n)$



- $\Rightarrow f(G_s) > f(n)$, therefore A* will never select G_s for expansion (i.e. cannot be an actual goal)
- $\Rightarrow$ **A* search is optimal**

Properties of A*

- Complete?
  - **Yes**, unless there are infinitely many nodes with $f \leq f(G)$
- Time?
  - $O(b^d)$ in the worst case, but it depends on how good the heuristic is
- Space?
  - $O(b^d)$ – keeps all nodes in memory
- Optimal?
  - **Yes** – cannot expand f_{i+1} until f_i is finished

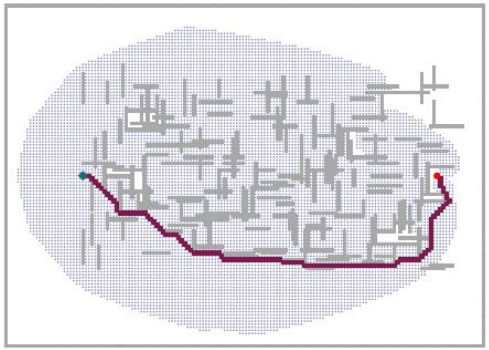
Weighted A*

- Heuristics that overestimate (inadmissible) → **weighted A***
- Might miss the optimal solution, but could be **more efficient**

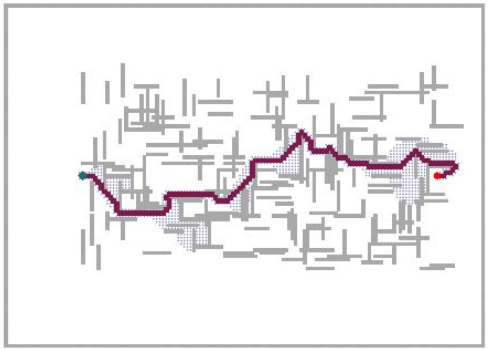
  ⇒ reduce number of expanded nodes

  $f(n) = g(n) + W \times h(n) \quad (1 < W < \infty)$

- E.g. in the figure
  - normal A* visits all the intermediate nodes (grey dots) to reach the goal
  - weighted A*, with $W = 2$, reduces the search space by a factor of 7, increasing the final cost by 5%



Normal A*



Weighted A*

Generalization of weighted A*

Weighted A* can be seen as a generalization of the others

- **Weighted A*** search:

  $g(n) + W \times h(n)$ $\quad (1 < W < \infty)$

- **A*** search:

  $g(n) + h(n)$ $\quad (W = 1)$

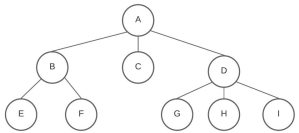
- **Uniform-cost** (**Dijkstra**) search:

  $g(n)$ $\quad (W = 0)$

- **Greedy best-first** search:

  $h(n)$ $\quad (W \to \infty)$

Quality of heuristic functions



- One way to characterize the quality of a heuristic is the **effective branching factor** b^*
- If the total number of nodes generated by A* for a particular problem is N and the solution depth is d, then b^* is the branching factor of a uniform tree of depth d containing $N+1$ nodes

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$$

- Experimental measurements of b^* on a small set of problems (e.g. in simulation) can provide a good guide to the heuristic's overall usefulness
- A well-designed heuristic would have a value of b^* close to 1

Example: 8-puzzle

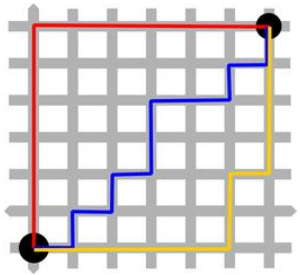
- Two possible heuristics:

  $h_1(n)$ = number of misplaced tiles

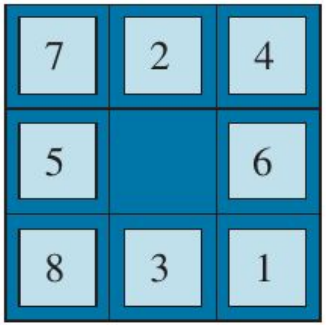
  $h_2(n)$ = total Manhattan distance

  (i.e., # of squares from desired location of each tile)

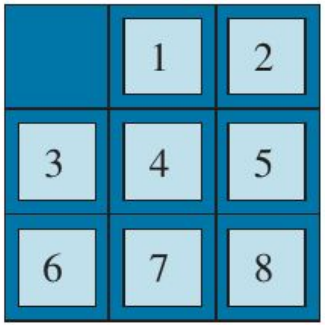
- $h_1(S)$ = ?
  - 8
- $h_2(S)$ = ? (for tiles 1 to 8)
  - 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18



Manhattan distance



Start State



Goal State

Example: 8-puzzle

	Search Cost (nodes generated)			Effective Branching Factor		
d	BFS	A*(h_1)	A*(h_2)	BFS	A*(h_1)	A*(h_2)
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Data averaged over 100 puzzles for each solution length d

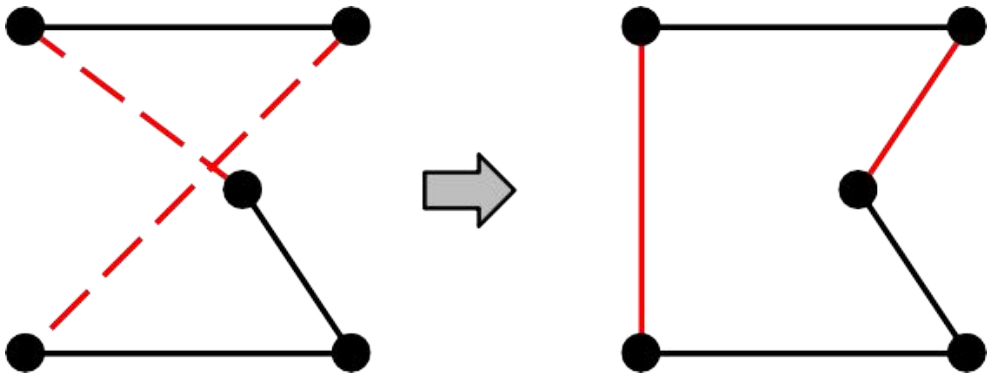
Local search and optimization problems

- In many optimization problems, path is irrelevant – the goal state itself is the solution
- State space = set of “complete” **configurations**
  - find optimal configuration, e.g. Travelling Salesperson Problem
  - or find configuration satisfying constraints, e.g. timetable
- In such cases, one can use **iterative improvement** algorithms
  - keep a single “current” state, try to improve it

- Local search algorithms operate by searching from a start state to neighboring states, without keeping track of the paths, nor the set of states that have been reached
- They might never explore a portion of the search space where a solution actually resides

Example: Travelling Salesperson Problem

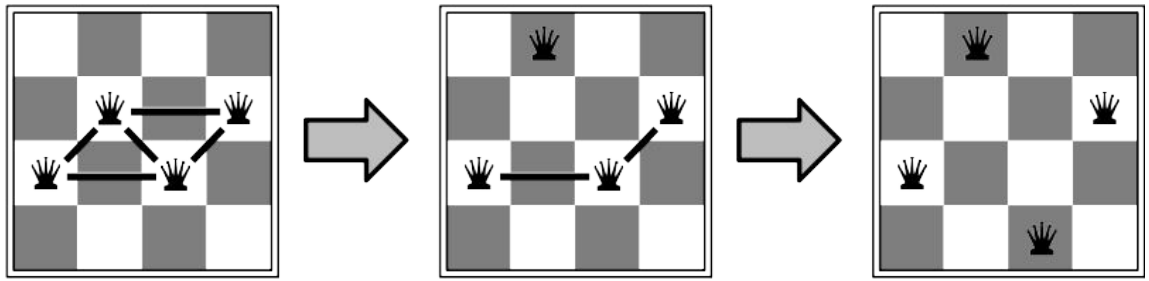
- Start with any complete tour, perform pairwise exchanges



- Local search methods can get within 1% of optimal very quickly with thousands of cities

Example: *n*-queens

- Put *n* queens on an *n* × *n* board with no two queens on the same row, column, or diagonal
- Move one queen at a time to reduce number of conflicts

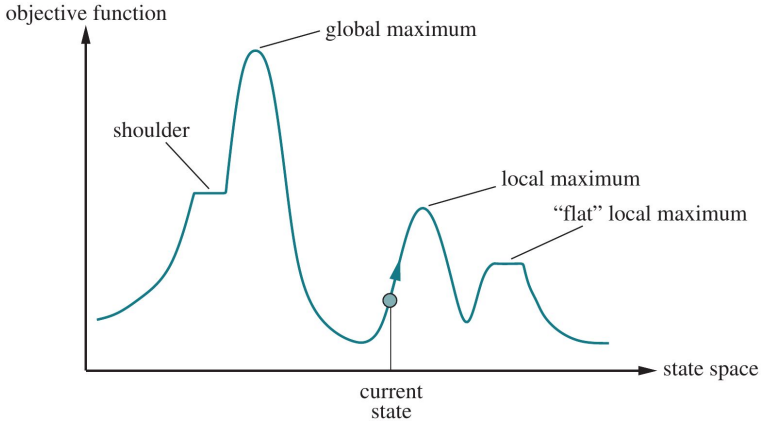


- *n*-queens problems can be solved almost instantaneously even for very large *n*!

Hill-climbing (also greedy local search)

- Useful to consider **state space landscape**

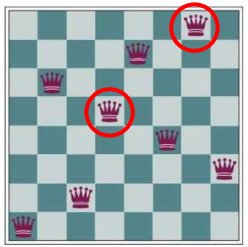
```
function Hill-Climbing( problem) returns a state that is a local maximum
    Inputs: problem, a problem
    local variables: current, a node
                     neighbor, a node

    current ← Make-Node(Initial-State[problem])
    loop do
        neighbor ← a highest-valued successor of current
        if Value[neighbor] ≤ Value[current] then return State[current]
        current ← neighbor
    end
```



- **Random-restart hill climbing**: overcomes local maxima – trivially complete
- **Random sideways moves**: 👍 escape from shoulders but 👎 loop on flat maxima (plateau)

Example: 8-queens problem

- An 8-queens state with heuristic cost estimate $h = 17$ (sum of the queens under attack in that configuration)
- The board shows the value of h for each possible successor obtained by moving a queen within its column
- There are 8 best moves with $h = 12$
- Hill-climbing will pick one of them to maximise $-h$ (negative of heuristic)



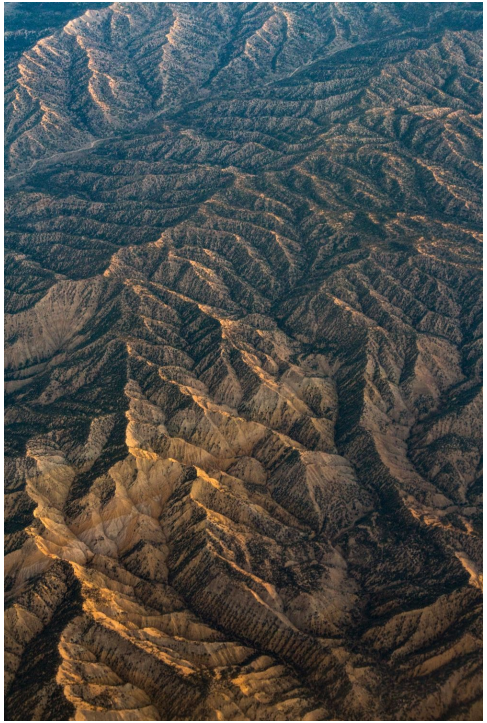
- Can get stuck in local maxima (e.g. figure with $h = 1$)

Simulated annealing

- Idea: escape local maxima by allowing some “bad” moves but gradually decrease their size and frequency
  - similar to hill-climbing, but now choosing random moves and accepting some bad ones (with probability p)
- The probability decreases with the “badness” $\Delta E(x)$ of the state x and a decreasing “temperature” T

$$p(x) \propto e^{-\Delta E(x)/T}$$

  - if T decreases slowly enough ⇒ always reach best state x^* with probability ~1
- Widely used in VLSI layout, airline scheduling, etc.



Simulated annealing: algorithm

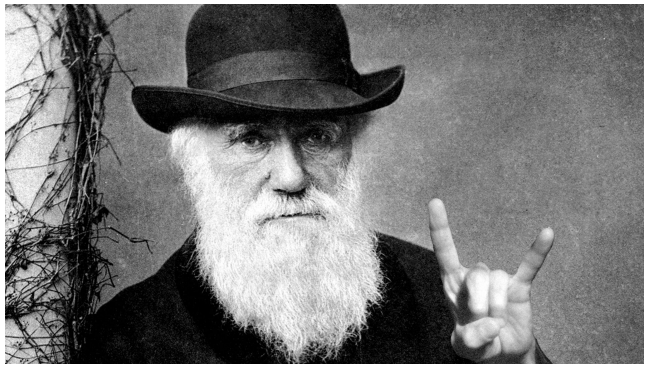
```
function Simulated-Annealing( problem, schedule) returns a solution state
    Inputs:          problem, a problem
                     schedule, a mapping from time to "temperature"
    local variables: current, a node
                     next, a node
                     T, a "temperature" controlling prob. of downward steps

    current ← Make-Node(Initial-State[problem])
    for t ← 1 to ∞ do
        T ← schedule[t]
        if T = 0 then return current
        next ← a randomly selected successor of current
        ΔE ← Value[next] – Value[current]
        if ΔE > 0 then current ← next
        else current ← next only with probability e^(-ΔE/T)
```

Local beam search

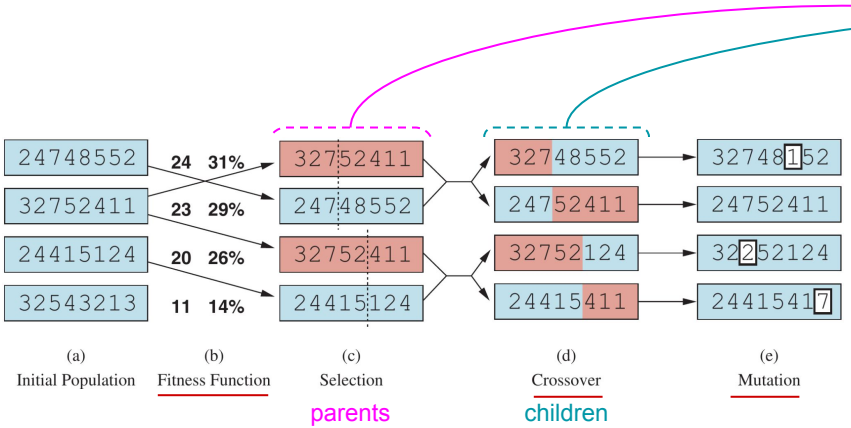
- **Idea**: start with *k* states (instead of 1) and continues with their best *k* successors
  - different from doing just *k* random restarts – each selection influences the next
  - searches that find good states recruit other searches to join them
- **Problem**: quite often, all *k* states end up on same local hill
- **Solution**: **stochastic beam search** chooses *k* successors randomly biased towards good ones
  - Close analogy to natural selection!

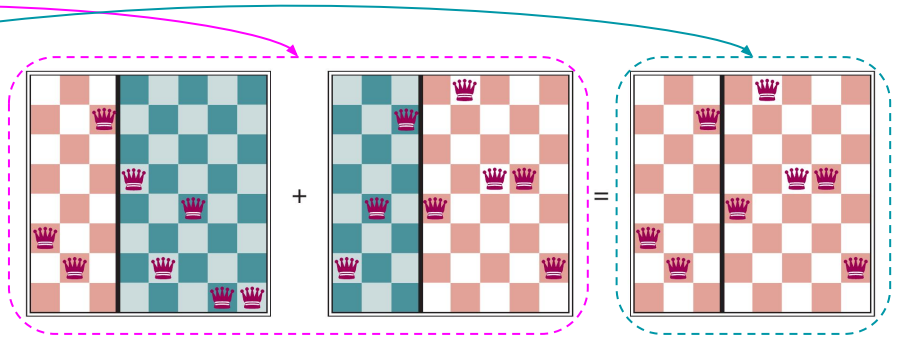
Genetic algorithms

- Similar to stochastic beam search + generate successors from pairs of states
- Inspired by natural selection in biology
  - there is a population of individuals (states), in which the fittest (highest value) individuals produce offspring (successor states) that populate the next generation
  - a process called **recombination**
- GAs require states encoded as **strings**
- Other approaches include
  - evolutions strategies
  - genetic programming



Genetic algorithms





- The **selection** process chooses the individuals who will become the parents of the next generation based on some **fitness function**
- The **recombination** procedure randomly select a **crossover point** to split each of the parent strings, and recombine the parts to form two children
- The final **mutation** randomly changes the offsprings depending on some **mutation rate**

Summary

- **Informed search** methods have access to a **heuristic** function that estimates the cost of a solution from the current state to a goal
  - Their performance depend on the quality of the heuristic function, possibly sacrificing optimality for speed
- **Local search** methods keep only a small number of states in memory that are useful for optimization
  - The only important thing is the goal, not the path to reach it

Conclusions

- Suggested reading

  Chapter 3, sec. 3.5-3.6

  Chapter 4, sec. 4.1 of Russell & Norvig

- Next lecture

  Logical agents

- Questions?